

Rapport — Étape 2 : comptes et connexion multi-clients

Application Fun Kids Animation · Corrections, vérifications et procédure de test · 24 septembre 2026

Élément	Détail
Étape du plan d'action	Étape 2 — Comptes et connexion multi-clients (retour client §4 et §10)
Branche / commit	nassim · commit du code : c99617d
Fichiers modifiés	users/models.py, users/forms.py, users/serializers.py, users/urls.py, users/admin.py, bookings/views.py, templates/auth/signup.html
Fichiers ajoutés	users/migrations/0003_normalize_user_emails.py, users/management/commands/prepare_test_accounts.py
Base de données	Migration de données 0003 — aucune table ni colonne modifiée (voir section 6)
Tests automatiques	100 tests : tous OK (79 existants + 21 nouveaux)
Résultat global	Étape 2 terminée

1. Objectif

Rendre fiable la connexion de plusieurs clients afin de pouvoir tester les scénarios multi-comptes demandés par le retour client (« la connexion avec le deuxième compte client ne fonctionne pas »). Objectifs précis :

- Permettre la connexion quelle que soit la casse de l'adresse e-mail saisie.
- Empêcher les doublons de comptes et les erreurs serveur à l'inscription, avec des messages clairs.
- Imposer un mot de passe robuste à l'inscription.
- Garantir que le rôle (Admin, Animateur, Client) ne puisse pas être modifié par l'utilisateur lui-même.
- Corriger l'incohérence de rôle du compte administrateur existant.
- Préparer les comptes de test Admin, Client 1, Client 2 et Animateur.
- Prouver que deux clients connectés en parallèle ne voient que leurs propres données.

2. Commits associés

Commit	Date	Message	Contenu
c99617d	24/09/2026 21:43	fix: connexion multi-clients et gestion des comptes (etape 2)	Corrections (7 fichiers), migration de données, commande de comptes de test, 21 tests
(suivant)	24/09/2026	docs: ajout du rapport PDF de l'etape 2	Ce rapport

Commit de référence pour le code : c99617d. Détail : `git show c99617d --stat`. Étape précédente : e8ed796 (sécurité).

3. Récapitulatif des corrections

#	Correction	Origine	Statut
1	Connexion insensible à la casse de l'e-mail (web et API)	Retour client §4	✓ Réalisé
2	Adresses enregistrées en minuscules + migration des comptes existants	Retour client §4	✓ Réalisé

#	Correction	Origine	Statut
3	Inscription : doublon refusé proprement, e-mail et mot de passe validés	Retour client §4	✓ Réalisé
4	Administration : doublon d'adresse différant par la casse refusé	Audit	✓ Réalisé
5	Rôle non modifiable par l'utilisateur via l'API	Découvert pendant l'étape	✓ Réalisé
6	Compte admin@funkidz.fr : rôle corrigé en ADMIN ; API des réservations alignée sur le statut staff	Audit	✓ Réalisé
7	Comptes de test Admin / Client 1 / Client 2 / Animateur	Retour client §4 et §10	✓ Réalisé
8	Isolation de deux clients connectés en parallèle, vérifiée	Retour client §4	✓ Réalisé

4. Description détaillée

4.1 Connexion insensible à la casse

Avant

La recherche du compte était sensible aux majuscules : « Nassim2389@hotmail.com » et « nassim2389@hotmail.com » étaient considérés comme deux adresses différentes. Un client qui saisisait son adresse avec une majuscule ne pouvait pas se connecter. C'est la cause la plus probable du problème de connexion du deuxième client.

Après

Toutes les adresses sont enregistrées en minuscules (création et chaque modification, quelle que soit la voie : inscription, administration, API, scripts). La connexion web et la connexion API (jeton) retrouvent le compte sans tenir compte de la casse ni des espaces autour de l'adresse.

Fichier : `users/models.py` (UserManager, User.save)

4.2 Inscription

Avant

Une adresse déjà utilisée provoquait une erreur serveur (page d'erreur 500). Aucun contrôle du format de l'adresse ni de la robustesse du mot de passe (« 123 » était accepté).

Après

Formulaire validé : format de l'e-mail, adresse déjà utilisée (message « Un compte existe déjà avec cette adresse e-mail »), règles de mot de passe de Django (longueur, mots de passe courants, trop numériques, trop proches de l'e-mail). Les erreurs s'affichent en français et l'adresse saisie est conservée. Mêmes règles pour l'API `/auth/register/`.

Fichier : `users/forms.py`, `users/urls.py`, `users/serializers.py`, `templates/auth/signup.html`

4.3 Protection du rôle

Avant

Le point d'API `/auth/me/` permettait à un utilisateur connecté de modifier son propre rôle (par exemple CLIENT → ADMIN) et d'obtenir ainsi l'accès à toutes les réservations.

Après

Le rôle est en lecture seule dans l'API : seule l'administration peut le modifier. L'API des réservations reconnaît aussi les comptes staff comme administrateurs.

Fichier : `users/serializers.py`, `bookings/views.py`

4.4 Comptes de test

Avant

Les adresses de test du retour client n'existaient pas dans la base, et le compte admin@funkidz.fr avait le rôle CLIENT alors qu'il était super-utilisateur (redirigé vers l'espace client au lieu de l'administration).

Après

Nouvelle commande prepare_test_accounts : crée ou met à jour les comptes de test avec le bon rôle, génère un mot de passe aléatoire par compte créé (affiché une seule fois, jamais stocké dans le code), ne modifie jamais le mot de passe d'un compte existant sauf demande explicite, et corrige le rôle des super-utilisateurs. Elle peut être relancée sans risque.

Fichier : users/management/commands/prepare_test_accounts.py

Comptes de test créés dans la base locale

Rôle	Adresse e-mail	État	Mot de passe
Admin	nassim2389@hotmail.com	créé	communiqué séparément
Client 1	nassimooouche@gmail.com	créé	communiqué séparément
Client 2	client2@funkidz.fr	créé	communiqué séparément
Animateur	animateur@funkidz.fr	existant	inchangé
Admin (existant)	admin@funkidz.fr	rôle corrigé → ADMIN	inchangé

Conformément au retour client, aucun mot de passe n'est inscrit dans ce document ni dans le dépôt Git.

5. Vérifications effectuées

5.1 Tests automatiques

Groupe de tests	Ce qui est vérifié	Résultat
Connexion (6)	E-mail stocké en minuscules, connexion web et API avec majuscules/espaces, mauvais mot de passe refusé	✓ Réalisé
Inscription (6)	Doublon avec autre casse, mot de passe faible, e-mail invalide, inscription valide, API register, formulaire admin	✓ Réalisé
Rôle (1)	Un client ne peut pas se déclarer ADMIN via /auth/me/	✓ Réalisé
Multi-clients (3)	Deux sessions parallèles isolées, API limitée à ses réservations, staff voit tout	✓ Réalisé
Commande (3)	Création des comptes et rôles, idempotence et mots de passe conservés, correction des super-utilisateurs	✓ Réalisé
Migration (2)	Passage en minuscules, aucune fusion de comptes en cas de collision	✓ Réalisé
Suite complète	100 tests, dont les 79 précédents (sécurité incluse)	✓ Réalisé

5.2 Essais sur le serveur réel

Serveur de vérification lancé en local sur **http://127.0.0.1:8011** (base du projet, après migration), puis arrêté après les essais.

Essai	Résultat obtenu	Statut
Client 1 se connecte en tapant « NassimOouche@Gmail.com »	302 vers l'accueil (connecté)	✓ Réalisé
Client 2 se connecte en même temps (autre session)	302 vers l'accueil (connecté)	✓ Réalisé

Essai	Résultat obtenu	Statut
Tableau de bord des deux clients	200 et 200, sessions distinctes	✓ Réalisé
/auth/me/ de chaque client	chacun reçoit uniquement son propre compte (rôle CLIENT)	✓ Réalisé
Admin se connecte avec « Nassim2389@hotmail.com »	302, puis /admin/ → 200	✓ Réalisé
Inscription avec « NASSIMOOUCHE@gmail.com » (déjà utilisée)	200 + « Un compte existe déjà avec cette adresse e-mail »	✓ Réalisé
Connexion avec un mauvais mot de passe	200 : formulaire réaffiché, pas de connexion	✓ Réalisé

6. Base de données — à signaler pour le rapport final

Élément	Détail
Migration	users/0003_normalize_user_emails — migration de données uniquement
Structure	Aucune table, colonne, relation ou contrainte ajoutée ou modifiée : le dictionnaire de données et l'UML restent valables
Effet	Les adresses e-mail existantes passent en minuscules ; en cas de doublon potentiel, le compte est laissé intact
À faire sur une autre machine	python manage.py migrate puis, pour les comptes de test, python manage.py prepare_test_accounts
Sauvegarde	Copie de la base locale faite avant migration, hors du dépôt (db.sqlite3.backup-avant-etape2)

7. Résultats obtenus

Sortie de la suite de tests complète sur le commit final :

```
$ venv/bin/python manage.py test
Found 100 test(s).
System check identified no issues (0 silenced).
Ran 100 tests in 108.230s

OK
```

Application de la migration et préparation des comptes sur la base locale :

```
$ venv/bin/python manage.py migrate users
Applying users.0003_normalize_user_emails... OK
$ venv/bin/python manage.py prepare_test_accounts
Comptes de test prêts :
Admin nassim2389@hotmail.com créé mot de passe : [masqué]
Client 1 nassimoouche@gmail.com créé mot de passe : [masqué]
Client 2 client2@funkidz.fr créé mot de passe : [masqué]
Animateur animateur@funkidz.fr existant mot de passe : (inchangé)
Rôle corrigé en ADMIN pour les super-utilisateurs : admin@funkidz.fr
```

Objectif	Résultat obtenu	Statut
Connexion quelle que soit la casse	« NassimOouche@Gmail.com » et « Nassim2389@hotmail.com » acceptés	✓ Réalisé
Inscription sans erreur serveur	Doublon → message clair ; mot de passe faible → refusé	✓ Réalisé
Rôle protégé	Tentative de passage CLIENT → ADMIN ignorée	✓ Réalisé

Objectif	Résultat obtenu	Statut
Admin cohérent	admin@funkidz.fr redirigé vers l'administration (rôle ADMIN)	✓ Réalisé
Comptes de test prêts	Admin, Client 1, Client 2, Animateur disponibles	✓ Réalisé
Isolation multi-clients	Deux sessions parallèles, chacune limitée à ses réservations	✓ Réalisé
Aucune régression	100/100 tests OK, étape 1 toujours respectée	✓ Réalisé

Conclusion : l'étape 2 est terminée et stable. Plusieurs clients peuvent désormais se connecter et être testés en parallèle, chacun limité à ses propres données.

8. Comment tester soi-même

8.1 Sur une nouvelle machine ou après récupération du code

```
venv/bin/python manage.py migrate
venv/bin/python manage.py prepare_test_accounts
```

Les mots de passe des comptes créés s'affichent une seule fois : les noter. Pour choisir un mot de passe commun aux comptes créés : `prepare_test_accounts --password <mot-de-passe>`. Pour réattribuer un mot de passe aux comptes existants : `ajouter --reset-passwords`.

8.2 Tests automatiques

```
venv/bin/python manage.py test users
```

8.3 Tests manuels (serveur lancé sur le port 8010)

```
venv/bin/python manage.py runserver 0.0.0.0:8010
```

- Navigateur 1 : `http://127.0.0.1:8010/auth/login/` → se connecter avec **NassimOouche@Gmail.com** (majuscules volontaires) : la connexion doit réussir.
- Navigateur 2 (ou fenêtre privée) : se connecter avec **client2@funkidz.fr** en même temps.
- Créer une réservation avec chaque client : chacun ne voit que la sienne dans « Mon espace ».
- Se connecter avec **Nassim2389@hotmail.com** : accès à l'administration (/admin/).
- Page d'inscription : essayer une adresse déjà utilisée, puis un mot de passe « 123 » : messages d'erreur clairs.

9. Suite du plan d'action

Étape	Contenu	Statut
1	Faibles de sécurité critiques (commit e8ed796)	✓ Réalisé
2	Comptes et connexion multi-clients (commit c99617d)	✓ Réalisé
3	Parcours de paiement Stripe (Link, échec, annulation)	À faire
4	Brevo et e-mails de bout en bout	À faire
5 à 10	Créneaux, administration, calendrier, livraison du 29/09, soutenance	À faire